

A Web-based Parcel Management System for a Courier Company

CS39440 Major Project Report

Author: Artem Diakov (ard38@aber.ac.uk)

Supervisor: Dr Emmanuel Isibor (emi3@aber.ac.uk)

30th April 2026

Version 2.0

This report is submitted as partial fulfilment of a BSc degree in
Computer Science (G400)



Department of Computer Science, Aberystwyth University,
Aberystwyth, Ceredigion, SY23 3DB, Wales, UK

Declaration of originality

I confirm that:

- This submission is my own work, except where clearly indicated.
- I understand that there are severe penalties for Unacceptable Academic Practice, which can lead to loss of marks or even the withholding of a degree.
- I have read the regulations on Unacceptable Academic Practice from the University's Academic Registry (AR) and the relevant sections of the current Student Handbook of the Department of Computer Science.
- I have read the document Generative AI and the Major and Minor Projects, which is available in the Blackboard module.
- In submitting this work, I understand and agree to abide by the University's regulations governing these issues.

Name: Artem Diakov

Date: 28/04/2026

Consent to share this work

By including my name below, I hereby agree to this project's report and technical work being made available to other students and academic staff of the Aberystwyth Computer Science Department.

Name: Artem Diakov

Date: 28/04/2026

Acknowledgements

I would like to express my gratitude to my project supervisor for their guidance and support throughout the development of this project. Their feedback helped shape both the technical implementation and the overall direction of the system as well as my second marker for the feedback during mid-project demonstration.

I would also like to thank the Department of Computer Science at Aberystwyth University for providing the resources and learning environment necessary to complete this project.

Finally, I would like to acknowledge the use of online documentation and development tools that supported the implementation of this system, including official resources for PHP, PostgreSQL, and third-party APIs such as Stripe and Twilio.

AI Declaration

This project made limited use of AI-assisted tools, including ChatGPT, to support the development process. These tools were primarily used to clarify technical concepts, assist with debugging, and provide guidance on implementation approaches.

All design decisions, system architecture, and final code were reviewed, adapted, and implemented independently. AI-generated suggestions were critically evaluated before inclusion to ensure correctness, security, and relevance to the project requirements.

The final system, report structure, and technical implementation represent my own work and understanding of the subject area.

Abstract

This project presents the design and implementation of ParcelPro, a web-based parcel management system developed for a local courier company. The system provides an integrated platform that supports parcel booking, payment processing, tracking, and internal management through a single application.

The application was implemented using PHP and PostgreSQL, with additional integrations including Stripe for secure online payments and Twilio for SMS notification handling. Customers can create orders as guests or registered users, receive delivery estimates, and track parcels using a unique reference number. Staff users can manage delivery progress, while administrators can control system data such as users, operating areas, and announcements.

The development process followed an iterative approach, allowing features to evolve through continuous implementation and testing. The final system demonstrates a complete end-to-end workflow, from order creation to delivery tracking, supported by validation, access control, and secure data handling.

While some limitations exist due to external service constraints and deployment conditions, the project successfully meets its objectives and demonstrates a functional and user-focused parcel delivery platform.

Table of Contents

Chapter 1 – Background, Requirements and Project Status	8
1.1 Introduction	8
1.2 Problem Definition	8
1.3 Requirements	9
1.4 Project Objectives.....	10
1.5 System Overview	11
1.6 Status of Work.....	12
1.7 Development Approach Summary	13
Chapter 2 – Research and Development Methodology.....	14
2.1 Courier Website Investigation.....	14
2.2 Design Implications	18
2.3 Technology Selection	19
2.4 Development Methodology	21
2.5 Security Guidance.....	21
Chapter 3 – System Design	23
3.1 System Architecture	23
3.2 Database Design.....	24
3.3 User Roles and Access Control.....	25
3.4 Main System Workflows.....	27
3.5 User Interface Design	30
3.6 Security Design.....	34
Chapter 4 – Implementation and Results	35
4.1 Homepage, Login and Registration.....	35
4.2 Parcel Booking Flow.....	36
4.3 Pricing, Delivery Estimates and Reference Numbers	37
4.4 Stripe Payment Integration	38
4.5 Tracking and Order History	38
4.6 Account Management.....	39
4.7 Staff Dashboard	39
4.8 Admin Dashboard.....	39
4.9 Twilio SMS Integration	41
4.10 Security Implementation	42

4.11 Deployment.....	43
4.12 Final Result.....	44
Chapter 5 – Testing.....	45
5.1 Testing Strategy	45
5.2 Functional Testing.....	45
5.3 Validation Testing	46
5.4 Security Testing.....	46
5.5 Example Test Cases	47
5.6 Testing Limitations.....	48
Chapter 6 – Critical Evaluation.....	49
6.1 Evaluation Against Requirements.....	49
6.2 Strengths.....	50
6.3 Weaknesses and Limitations.....	51
6.4 SWOT Analysis.....	52
6.5 Future Work	52
Chapter 7 – Conclusion.....	54
Annotated References.....	55
Appendix A – Use of Third-Party Code and Libraries.....	56
Appendix B – Database Tables	58
Document Change History.....	63

Chapter 1 – Background, Requirements and Project Status

1.1 Introduction

The aim of this project was to design and implement a web-based parcel management system for a local courier company. The system was intended to support the main activities involved in arranging parcel collection and delivery, including customer order creation, online payment, parcel tracking, and internal staff management.

Parcel delivery services usually involve several connected processes. A customer needs to provide collection and delivery details, the system needs to confirm whether the company can serve those locations, payment needs to be completed, and the parcel must then move through a set of delivery stages. If these activities are handled manually or across separate tools, the process can become slow, inconsistent, and difficult to manage.

The system developed for this project, named ParcelPro, was designed to bring these activities into one web application. Customers can create orders as guests or registered users, track parcels using a reference number, and receive delivery updates. Staff users can search for orders and update tracking progress, while administrative users can manage accounts, operating areas, and announcements.

The project was developed using PHP, PostgreSQL, JavaScript, Bootstrap, Apache, Stripe, and Twilio. The final system was deployed on a university virtual machine and made accessible over HTTPS. Although some limitations remain, particularly around external service constraints and account deletion, the submitted system demonstrates a complete parcel delivery workflow from booking to tracking.

1.2 Problem Definition

The problem addressed by this project is that a small courier company needs a reliable way to manage parcel orders without relying on disconnected manual processes. In a basic workflow, customers may contact the company directly, staff may manually check whether an address is supported, and delivery progress may be recorded informally. This can lead to mistakes, delays, and poor visibility for customers.

A more useful system needs to support both customer-facing and internal operations. Customers should be able to place an order clearly, understand the price before paying, and track the order afterwards. At the same time, staff need a controlled way to update parcel progress, and administrators need to manage the areas where the company operates.

The system also needs to reject orders that fall outside approved geographical areas. Without this, customers could place orders that the courier company cannot actually collect or deliver. This was one of the main requirements of the project and directly shaped the database design and the parcel booking process.

Another part of the problem is security. A parcel system handles personal details such as names, addresses, phone numbers, emails, and account credentials. It also connects to payment and SMS services. Therefore, the system needed to be designed with input validation, access control, session management, and safe database handling from the beginning.

1.3 Requirements

The original project brief defined the system as a web platform for managing the collection and delivery of parcels for a local courier company. The brief included several indicative tasks, which were converted into functional requirements for the implemented system.

ID	Requirement	Status
FR1	Customers can register for an account.	Complete
FR2	Customers can place orders as registered users or guests.	Complete
FR3	Customers can submit parcel collection and delivery details.	Complete
FR4	The system accepts or rejects orders based on approved geographical areas.	Complete
FR5	Each order has an estimated delivery date.	Complete
FR6	Customers can pay online.	Complete
FR7	Customers can track parcels.	Complete
FR8	Staff can update parcel progress.	Complete
FR9	Admin users can manage users, staff, operating areas, and announcements.	Complete

FR10	SMS notifications are integrated for order and delivery updates.	Partial
FR11	Security controls protect user data and restricted functionality.	Complete

The SMS requirement is marked as partial because the Twilio integration was implemented and SMS attempts are logged in the database, but delivery is restricted by Twilio trial and routing limitations. The system captures provider responses and errors, so the technical integration is present, but reliable delivery would require a production Twilio configuration.

1.4 Project Objectives

The main objective was to produce a working web application that demonstrates the full parcel delivery process. This included both the customer journey and the operational tools required by staff and administrators.

The specific objectives were:

- to design a user-friendly homepage where users can begin sending or tracking a parcel quickly;
- to implement a multi-step parcel booking process with strong validation;
- to check sender and recipient postcodes against approved operating areas;
- to calculate a delivery price and estimated delivery date;
- to integrate Stripe payment in test mode;
- to generate a unique tracking reference for each paid order;
- to provide a tracking page that displays order progress and tracking history;
- to provide registered users with order history and account management;
- to provide staff with tools to search orders and update parcel status;
- to provide administrators with tools to manage users, staff, operating areas, and announcements;
- to integrate SMS notification handling using Twilio;
- to deploy the system on a university virtual machine.

These objectives were used later in the project to evaluate whether the final system met the original brief.

1.5 System Overview

ParcelPro is split into three main user areas: customer functionality, staff functionality, and administrative functionality.

Customers and guests interact with the public side of the system. They can use the homepage to start a parcel booking or track an existing order. The parcel booking process collects contact details, sender details, recipient details, parcel dimensions, and delivery options. Once the order has been reviewed and paid for, the system creates the order and generates a tracking reference.

Registered customers have additional account features. They can view their order history, open order details, update profile information, change their password, and save one default address. The saved address can then be reused during the parcel booking flow to reduce repeated input.

Staff users have access to a dashboard that focuses on operational order management. They can search and filter orders, view detailed parcel information, and update tracking progress. Tracking updates include status, location, description, timestamp, and the user who made the update.

Administrators have access to wider system management. They can manage customer accounts, staff accounts, operating areas, and homepage announcements. Administrators also have additional control over order states, such as the ability to cancel orders.

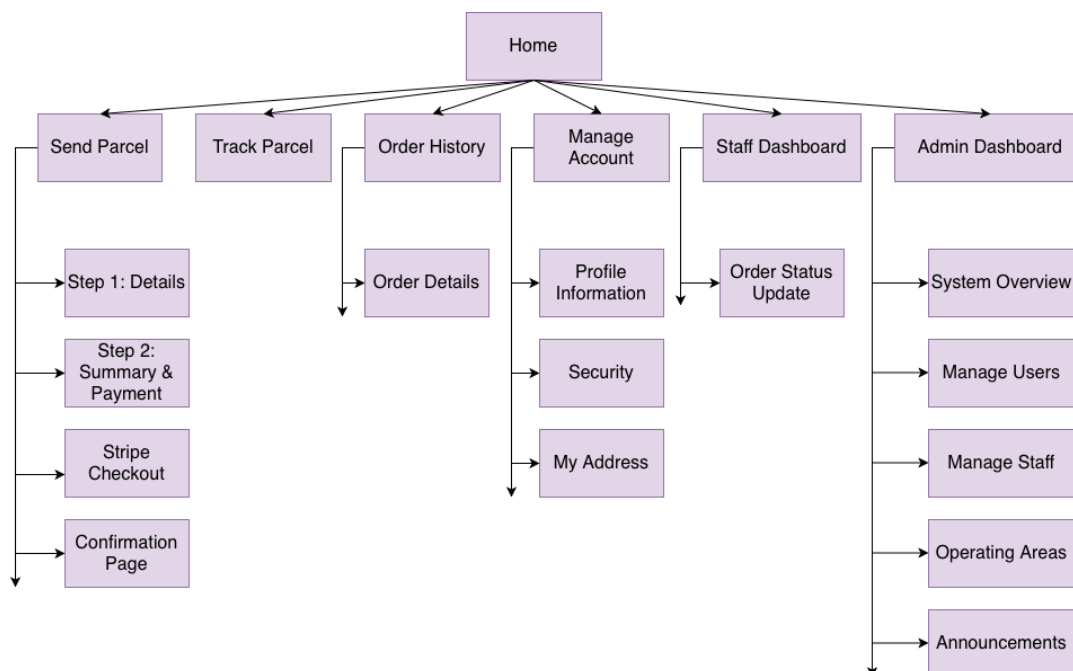


Figure 1 shows the overall sitemap of the system.

1.6 Status of Work

The table below summarizes the status of the main project activities at the point of submission.

Activity	Status	Notes
Customer registration and login	Complete	Includes password hashing, validation, and session handling.
Guest parcel booking	Complete	Users can place orders without creating an account.
Registered user parcel booking	Complete	Logged-in users can reuse saved sender address details.
Operating area validation	Complete	Sender and recipient postcodes are checked against database records.
Price calculation	Complete	Price includes base, weight, size, quantity, and collection charge.
Estimated delivery date	Complete	Calculated using delivery type and working days.
Stripe payment integration	Complete	Implemented using Stripe test mode.
Order confirmation and reference generation	Complete	Unique references are generated after successful payment.
Parcel tracking	Complete	Orders can be tracked using a reference number.
Order history	Complete	Available for registered users.
Account management	Complete	Users can update profile, password, and saved address.
Staff dashboard	Complete	Staff can search orders and update tracking progress.

Admin dashboard	Complete	Admins can manage users, staff, areas, and announcements.
SMS notification integration	Partial	Twilio integration exists, but delivery is limited by provider restrictions.
Deployment	Complete	Deployed on university VM using Apache and HTTPS.
Account deletion	Not started	Identified as future work due to GDPR/user self-service considerations.
Password reset	Not started	Future work requiring email or SMS verification flow.

1.7 Development Approach Summary

The project followed an iterative development approach inspired by Agile and Scrum. A fully formal Scrum process was not suitable because this was an individual project, but the core idea of working in small cycles was useful.

Development was carried out by choosing a feature or problem area, implementing it, testing it manually, and then refining it before moving to the next feature. For example, the parcel booking form was first developed as a basic form, then improved with progressive sections, validation, postcode checks, session storage, and review/edit behaviour.

This approach was useful because several parts of the project became clearer during implementation. Payment integration, SMS delivery, staff status updates, and validation behaviour all required adjustments as the system became more complete. The iterative approach allowed these changes to be handled without needing to redesign the whole system at once.

A more detailed discussion of the development methodology is provided in Chapter 2.

Chapter 2 – Research and Development Methodology

2.1 Courier Website Investigation

To understand how modern parcel delivery platforms are designed, a comparative investigation was conducted on three widely used courier services: Royal Mail, Evri, and DPD. These systems were selected due to their prevalence in the UK and their differing approaches to usability, workflow structure, and feature integration.

Royal Mail presents a structured and professional interface, with clearly separated stages for sending parcels. As shown in *Figure 2*, users are immediately presented with sending and tracking options, supported by visible pricing tiers and service descriptions. However, the ordering process is distributed across multiple pages, requiring users to move between stages such as destination, parcel details, and payment. This multi-page approach increases clarity but introduces friction in the user journey

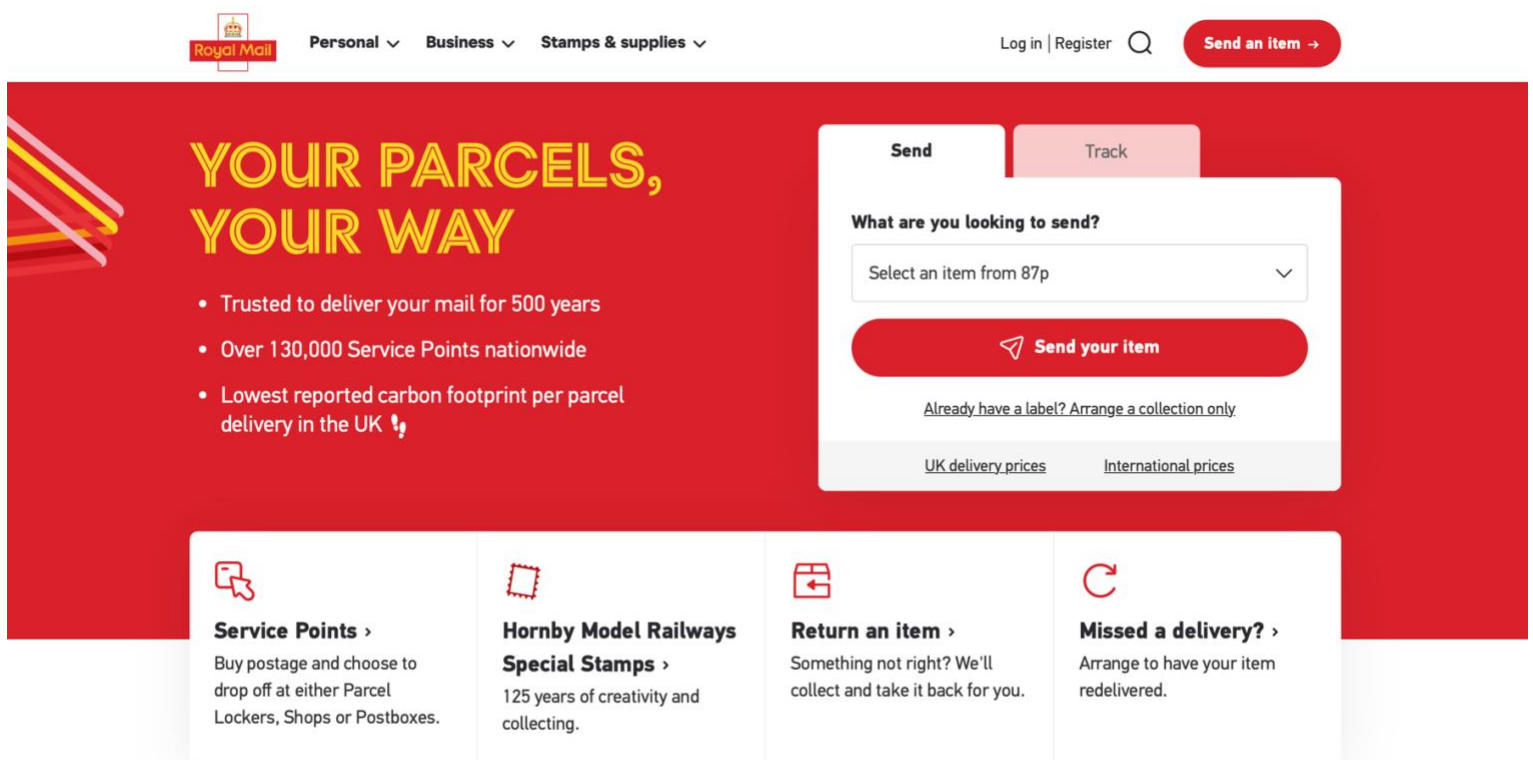


Figure 2: Royal Mail Homepage

Evri adopts a significantly different approach, focusing on simplicity and task-oriented interaction. As shown in *Figure 3*, the homepage features a combined “Send / Track / Return” interface, allowing users to begin key actions immediately without navigating away from the page.

The screenshot displays the Evri homepage with a blue header. The header includes navigation links for 'Personal', 'Business', and 'News & Corporate' on the left, and 'Log in' and 'Sign up' on the right. Below the header, the Evri logo is followed by 'Send', 'Track', and 'Return' buttons. To the right are links for 'Our Services', 'ParcelShops', 'Careers', and 'Help'. A Trustpilot rating of 'Great' with 5,297,420 reviews is shown. The main content area has a blue background with the heading 'A great value parcel delivery & courier service'. Below this is a 'Send' form with tabs for 'Send', 'Track', and 'Return'. The form includes a 'FREE end-to-end tracking with every parcel' banner, a 'SEND from £2.62 (£2.18 + VAT)' offer, and a prompt to 'Choose a ParcelShop delivery for your cheapest way to send to the UK'. The form fields are 'Destination country' (set to 'United Kingdom'), 'From postcode*', 'To postcode*', and 'Weight (kg)' (set to 'Please select'). A 'Send a parcel >' button is at the bottom right. Below the form are links for 'What you can and can't send' and 'Parcel size and weight guide'. The footer contains four promotional cards: 'Global shipping New' (Send international parcels from £6.08), 'Up to 50% savings' (Get exclusive discounts when you send 40+ parcels a month), 'Bulk send goes global' (Link your marketplaces & manage UK & international orders in one place), and 'Drop off at any ParcelShop' (9,000+ ParcelShops to choose from across the UK). A wheelchair icon is in the bottom right corner.

Figure 3: Evri Homepage

This design reduces friction and prioritises user intent. Additionally, the ordering process is implemented as a progressive form, where sections are revealed only after previous inputs are completed. This behaviour is illustrated in *Figure 4*, where selecting delivery options unlocks subsequent steps.

delivery made for you

Send > **Delivery Options** > What's in it? > Where's it going? > Your details > Payment

Your delivery options

Delivery speed

Standard **£2.62**

Our usual great service, delivery in 2 - 4 working days.

Choose from ParcelShop or Locker drop off or Courier Collection.

Your nearest drop off is [0.2 miles away](#)

☒ **Selected**

Next Day **£3.20**

Delivery in 1 working day to most UK destinations.

For the best chance of Next Day delivery, drop off your parcel by 12 noon.

ParcelShop and Locker drop off only

Your nearest drop off is [0.2 miles away](#)

☐ **Select**

Drop off or collection

ParcelShop or Locker Drop off **£2.62**

Delivery usually in 2 - 3 working days from drop off.

Summary

From: YO164NQ **To:** YO149AT

Sending restrictions:

You can send parcels up to **120cm** long and with a combined length and circumference of **245cm**.

[View parcel size limits](#)

Nearest drop off location:

ONE STOP, 182-184 Quay Road,
Bridlington, YO16 4JE, GB

You can drop your parcel off at any of our ParcelShops or Lockers without changing your selection

Free Cover up to £20	Free
Total (inc VAT)	£2.62



Figure 4: Evri Progressive Form Behaviour

This progressive interaction model was identified as highly effective in reducing cognitive load and guiding users through the process.

DPD provides a more structured and data-driven system. As shown in *Figure 5*, the booking process follows a step-by-step workflow supported by a clear progress indicator.

Unlike Evri, DPD requires users to manually input detailed parcel information such as dimensions and value, which increases accuracy but also increases complexity. However, DPD excels in its tracking capabilities, providing detailed delivery history with timestamps and location updates.

The screenshot displays the DPD booking interface. At the top, a 'County' dropdown menu is set to 'North Yorkshire'. Below this, a section titled 'How big is your parcel?' contains several input fields: 'Weight (kg) *' with a value of '1', 'Height (cm) *', 'Width (cm) *', 'Length (cm) *', 'Quantity *', and 'Value (£) *'. Each field has a maximum value indicated (e.g., 'Maximum parcel weight 30kg'). A link 'VIEW MAXIMUM PARCEL SIZES' is provided. At the bottom of this section is a red button '+ ADD ANOTHER PARCEL'. To the right, a 'Your Summary' sidebar shows 'VAT 0.00' and 'Total (inc. VAT) 0.00'. It also displays logos for 'G Pay', 'Apple Pay', 'PayPal', 'VISA', and 'MasterCard'. A 'NEXT STEP' button is at the bottom of the sidebar. Below the parcel details, a section titled 'Where should we deliver your parcel?' offers two options: 'Direct to Pickup Point' (marked 'Most popular') and 'Deliver to the recipient'.

Figure 5: DPD Structured Booking and Detailed Input Requirements

Overall, the investigation revealed three key patterns:

- Evri – simplicity and speed
- Royal Mail – structure and clarity
- DPD – detailed tracking and communication

These observations formed the foundation for the design decisions implemented in ParcelPro.

2.2 Design Implications

The findings from the investigation were directly applied to the design of ParcelPro. Rather than replicating a single system, the goal was to combine the strongest elements of each platform while avoiding their limitations.

The most significant influence came from Evri's homepage design. ParcelPro adopts a similar hero section containing both "Send" and "Track" functionality, allowing users to immediately begin key actions without navigating through menus.

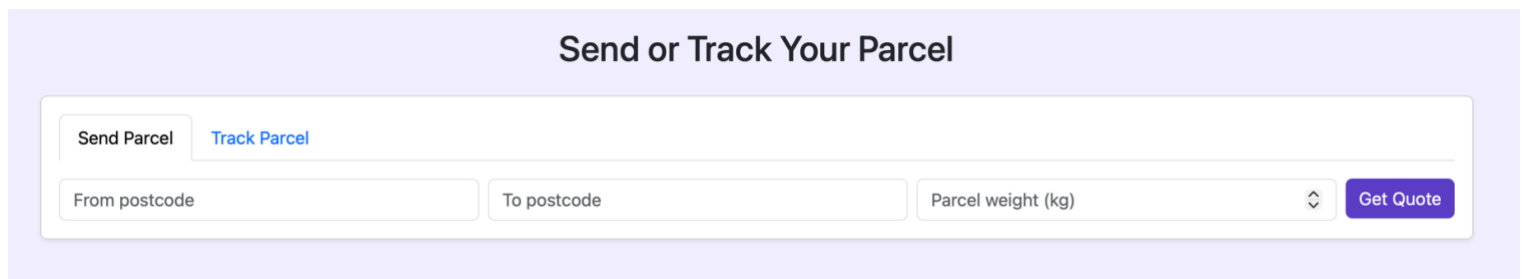


Figure 6: ParcelPro Homepage

This decision was made to prioritise usability and reduce unnecessary navigation steps.

Another major design decision was the implementation of a progressive multi-section form. While Evri dynamically reveals sections, ParcelPro takes a slightly different approach – all sections are visible from the start but remain locked until prerequisites are completed.

This design was chosen for two reasons:

- It maintains clarity of the overall process, allowing users to see what information will be required.
- It still enforces a structured flow by preventing interaction with incomplete sections.

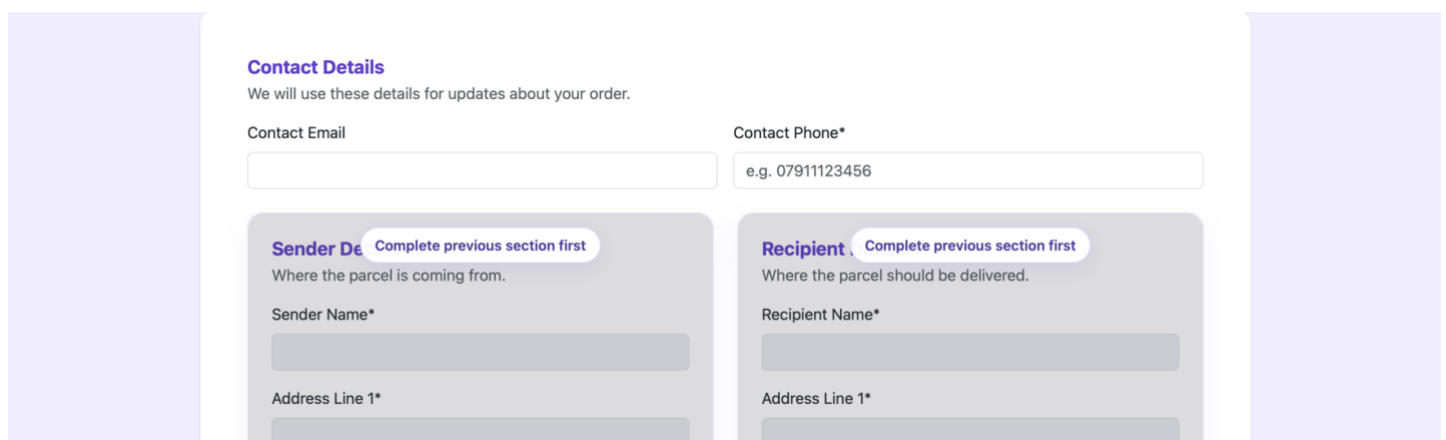


Figure 7: Progressive Form with Locked Sections

Contact Details
We will use these details for updates about your order.

Contact Email

Contact Phone*

Sender Details
Where the parcel is coming from.

Sender Name*

Address Line 1*

Recipient Complete previous section first
Where the parcel should be delivered.

Recipient Name*

Address Line 1*

Figure 8: Progressive Form with Unlocked Section

This represents a refinement of Evri’s approach, combining visibility with controlled progression.

From DPD, the concept of detailed tracking history was adopted. ParcelPro includes a full tracking log with timestamps and status updates, ensuring transparency for both customers and staff.

Tracking History			
Date & Time	Location	Description	Updated By
27/04/2026 22:56	Filey	Order placed and payment confirmed.	System

Figure 9: Parcel Tracking History in ParcelPro

Additionally, DPD’s use of SMS notifications for delivery updates influenced the integration of Twilio within the system. Although full delivery is limited by provider restrictions, the system implements complete logging of SMS attempts.

2.3 Technology Selection

The technology stack was chosen to keep the project practical, maintainable, and suitable for the available development time. The aim was not to use the most complex technologies, but to select tools that could support the required features reliably.

PHP was selected for the backend because it is well suited to server-side web development and works naturally with form-based applications. The system required a large amount of request handling, input validation, session management, and database access. PHP allowed these parts to be implemented directly without needing a heavy framework. This was useful for an individual project because it made the application easier to understand, debug, and deploy.

PostgreSQL was selected as the database system because the project required structured and related data. The application stores users, orders, payments, tracking history, operating areas, addresses, announcements, and SMS notification logs. A relational database was suitable because these records are connected. For example, an order can have a payment record, multiple tracking history records, and an optional registered user. PostgreSQL also provides strong support for constraints and data integrity, which helped keep the system reliable.

Bootstrap was used for the user interface to provide a consistent responsive layout. This was important because the application needed to be usable on different screen sizes without spending excessive time building all styling manually. Custom CSS was still used where needed, especially for the parcel booking form, progress indicators, dashboard cards, and tracking layout.

JavaScript was used to improve usability on the client side. It supports the progressive send form, validation feedback, postcode checks, locked sections, and button enabling behaviour. However, JavaScript validation was not treated as the main security control. It was used to improve the user experience, while PHP validation remained responsible for enforcing the rules on the server.

Apache was used as the web server on a Debian-based university virtual machine. This provided a realistic deployment environment compared with running only on localhost. The final deployment is available through the university VM using HTTPS at <https://debproj-ard38.teachvm.aber.ac.uk/index.php>, with access limited to approved users through the university network or VPN.

Stripe was selected for payment processing because it provides a hosted checkout flow. This meant card details are entered on Stripe's side rather than being stored or processed directly by ParcelPro. This reduced the security burden of the project while still allowing online payment to be demonstrated properly in test mode.

Twilio was selected for SMS notification handling because it provides an API that can be called from the backend when order events occur. The system logs SMS attempts in the database, including provider responses and errors. In the submitted version, SMS delivery is limited by

provider restrictions, but the integration still demonstrates how the notification flow would operate in a production setup.

2.4 Development Methodology

The development process followed an adapted Agile approach, influenced mainly by Scrum. Scrum had been used in a previous group project, so its structure was familiar. However, this project was completed individually, which meant that a full Scrum process with formal roles, sprint meetings, and team ceremonies would not have been appropriate.

Instead, the project used the parts of Scrum that were useful for an individual development process. Work was divided into smaller feature areas, such as authentication, parcel booking, payment, tracking, staff tools, and admin management. Each feature was implemented, tested manually, reviewed, and then improved before moving on to the next major feature.

This approach suited the project because the requirements became clearer as the system developed. For example, the parcel booking form originally started as a simpler form, but later became a multi-section flow with locked panels, postcode validation, saved sender details, review/edit behaviour, and Stripe payment. Similarly, the staff dashboard developed from a basic order list into a more structured operational tool with status filters, tracking history, controlled status updates, and restricted cancellation logic.

The iterative approach also helped when external services were added. Stripe payment integration required testing the full redirect flow between ParcelPro and Stripe Checkout. Twilio SMS integration also required changes after provider restrictions were discovered. Because the project was developed in stages, these issues could be handled without needing to restart the whole design.

This methodology provided a balance between planning and flexibility. The main requirements were known from the start, but the exact interface, validation behaviour, and dashboard design evolved during implementation.

2.5 Security Guidance

Security was considered during both design and implementation because the system handles personal data, account credentials, delivery addresses, payments, and staff/admin functionality. The main security approach was to apply several smaller controls together rather than relying on one single protection.

Input validation was applied at multiple levels. HTML validation and JavaScript validation provide immediate feedback to users, but server-side PHP validation is the final enforcement point. This is important because client-side checks can be bypassed. For example, parcel details, phone numbers, email addresses, names, postcodes, delivery options, and account details are all checked again on the server before being accepted.

Database security was handled using parameterised queries. User input is not concatenated directly into SQL statements, which reduces the risk of SQL injection. This was important across the system because many pages accept user input, including login, registration, parcel booking, tracking search, account management, staff search, and admin forms.

Dynamic output is escaped before being displayed in the browser. This reduces the risk of cross-site scripting, where malicious content could otherwise be rendered as HTML or JavaScript. This was especially important for fields such as names, addresses, announcements, and tracking descriptions, because those values come from users or staff.

Authentication uses PHP's password hashing API. Passwords are stored as hashes rather than plain text, and login checks are performed using *password_verify()*. A peppered password design was considered, but was not adopted because it would add secret-management complexity. For this project, PHP's standard password hashing approach was considered more maintainable and appropriate.

Session handling was also considered. After a successful login, the session ID is regenerated to reduce the risk of session fixation. Sessions are also used to hold temporary order data during the parcel booking process. This data is cleared once the order has been completed.

CSRF protection is used on sensitive forms. Each form includes a token linked to the session, and the server rejects requests where the token is missing or invalid. This helps prevent unwanted actions being submitted from another site.

Access control is enforced on the server side through role checks. Staff and admin pages cannot be accessed simply by typing the URL. Staff users can update parcel progress, while admin users have additional permissions such as managing users, operating areas, announcements, and cancellation states.

Overall, the security design focused on practical protections against common web application risks while keeping the implementation understandable and maintainable.

Chapter 3 – System Design

3.1 System Architecture

The system follows a standard web application architecture consisting of a client-side interface, a server-side application, a relational database, and external service integrations.

Users access the system through a web browser. The client-side layer is responsible for presenting the interface and collecting user input. It is implemented using HTML, CSS, JavaScript, and Bootstrap. This layer provides immediate feedback through validation and controls how the user progresses through processes such as parcel booking and account management.

The server-side application is implemented in PHP and handles all core logic of the system. This includes processing user requests, validating input, managing sessions, performing database operations, and controlling access to restricted functionality. The backend also acts as the integration layer for external services, including Stripe for payment processing and Twilio for SMS notifications.

All persistent data is stored in a PostgreSQL database. The database manages users, orders, payments, tracking history, operating areas, and other supporting data. Relationships between these entities ensure that the system maintains consistency across the full parcel lifecycle.

External services are integrated into the system to extend functionality. Stripe is used to handle secure payment processing through a hosted checkout, while Twilio is used to manage SMS notifications. These services are accessed through API calls made from the server-side application.

The system is deployed on a Debian-based virtual machine using the Apache web server. This allows the application to be accessed remotely and simulates a real-world deployment environment rather than a local-only setup.

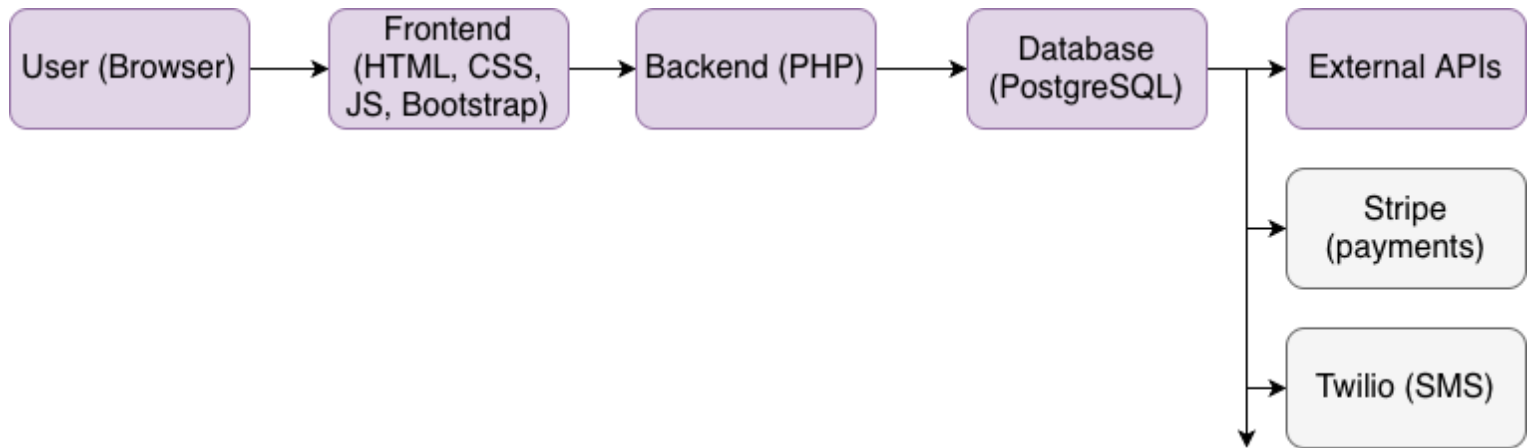


Figure 10: System Architecture

3.2 Database Design

The system uses a relational database implemented in PostgreSQL to manage all application data. The design is centred around the lifecycle of a parcel order, from creation and payment to tracking and delivery.

The [*users* table](#) stores account information, including name, email, phone number, password hash, and role. Roles are restricted to predefined values such as customer, staff, and admin, enabling role-based access control throughout the system.

The [*orders* table](#) is the core entity and contains all parcel-related information. This includes sender and recipient details, parcel dimensions, pricing information, delivery type, status, and a unique reference number. Each order represents a complete parcel transaction within the system.

The [*payments* table](#) stores payment-related data and is linked directly to orders. This ensures that each order has a corresponding payment record and that only successfully paid orders are stored.

The [*tracking_history* table](#) records all status updates for an order. Each record includes the status, location, timestamp, and the user who performed the update. This provides full traceability of parcel movement and supports the tracking functionality visible to both customers and staff.

The [*operating_areas* table](#) defines valid service postcodes. These are used during parcel booking to ensure that orders can only be placed within supported areas. The same data is also used for selecting tracking locations in staff updates.

Additional supporting tables include:

- [user_addresses](#) for storing saved sender addresses,
- [announcements](#) for system messages displayed to users,
- [sms_notifications](#) for logging SMS attempts and provider responses,
- [collection_points](#) for potential drop-off locations.

The database design ensures that relationships between entities are enforced and that invalid data is minimised. For example, tracking records cannot exist without an associated order, and payments are linked directly to a valid order reference.

A full definition of database tables, including column-level descriptions, is provided in Appendix B.

3.3 User Roles and Access Control

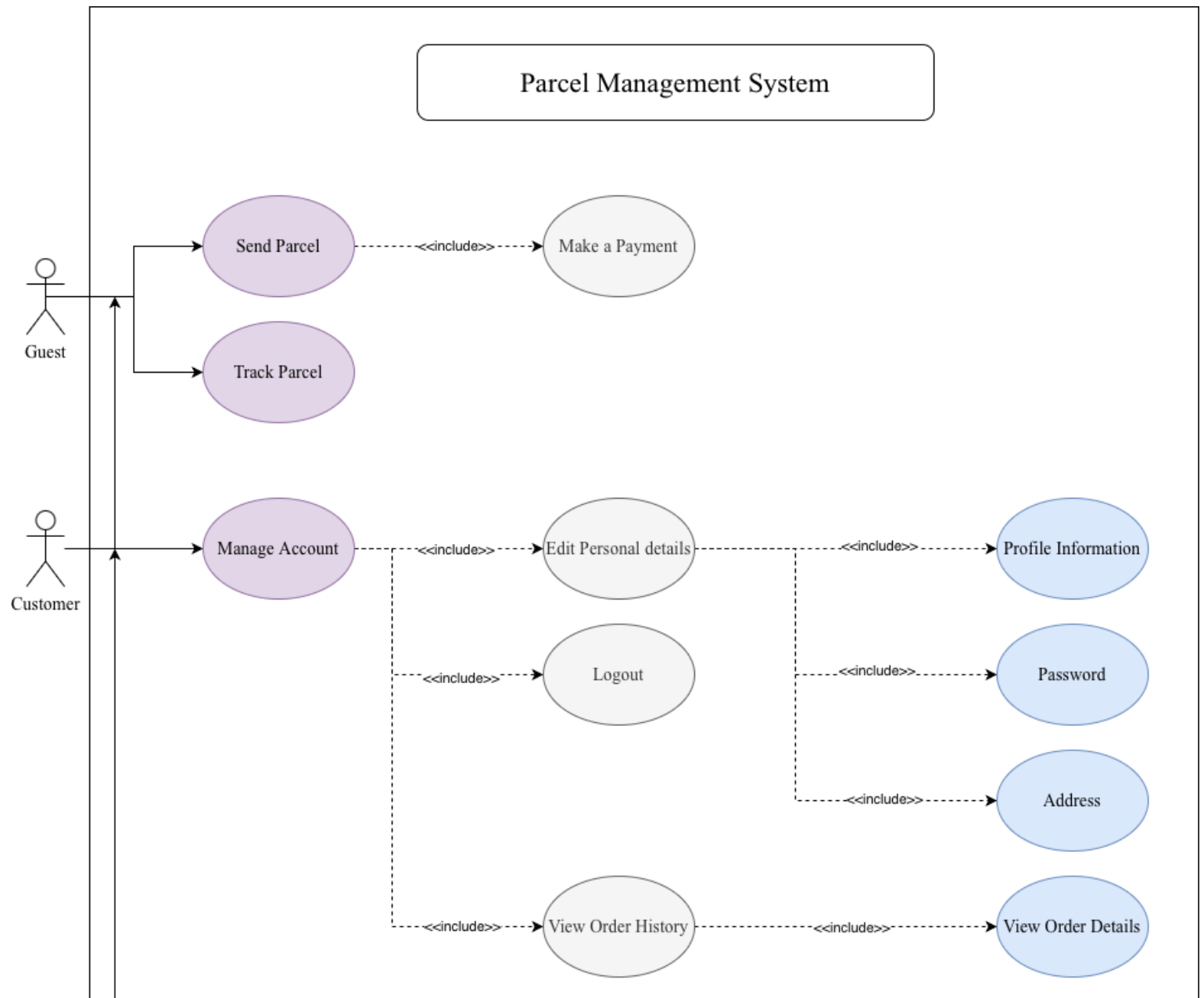
The system is designed around three main user roles: customer, staff, and administrator. Each role has access to a different set of functionality, and access control is enforced on the server side.

Customers can create and track parcel orders. They may use the system either as guests or as registered users. Registered users have additional functionality, including order history, account management, and saved address details.

Staff users have access to operational tools for managing parcel delivery. This includes searching for orders, viewing parcel details, and updating tracking progress. Staff users are restricted from performing administrative actions such as managing user roles or cancelling orders.

Administrators have full access to system management functionality. They can manage user accounts, assign roles, control operating areas, and create system announcements. Administrators also have additional control over order states, including the ability to cancel orders where necessary.

Access control is implemented using role checks within the backend. Restricted pages cannot be accessed directly without appropriate permissions, even if the URL is known. This ensures that sensitive operations are protected against unauthorised access.



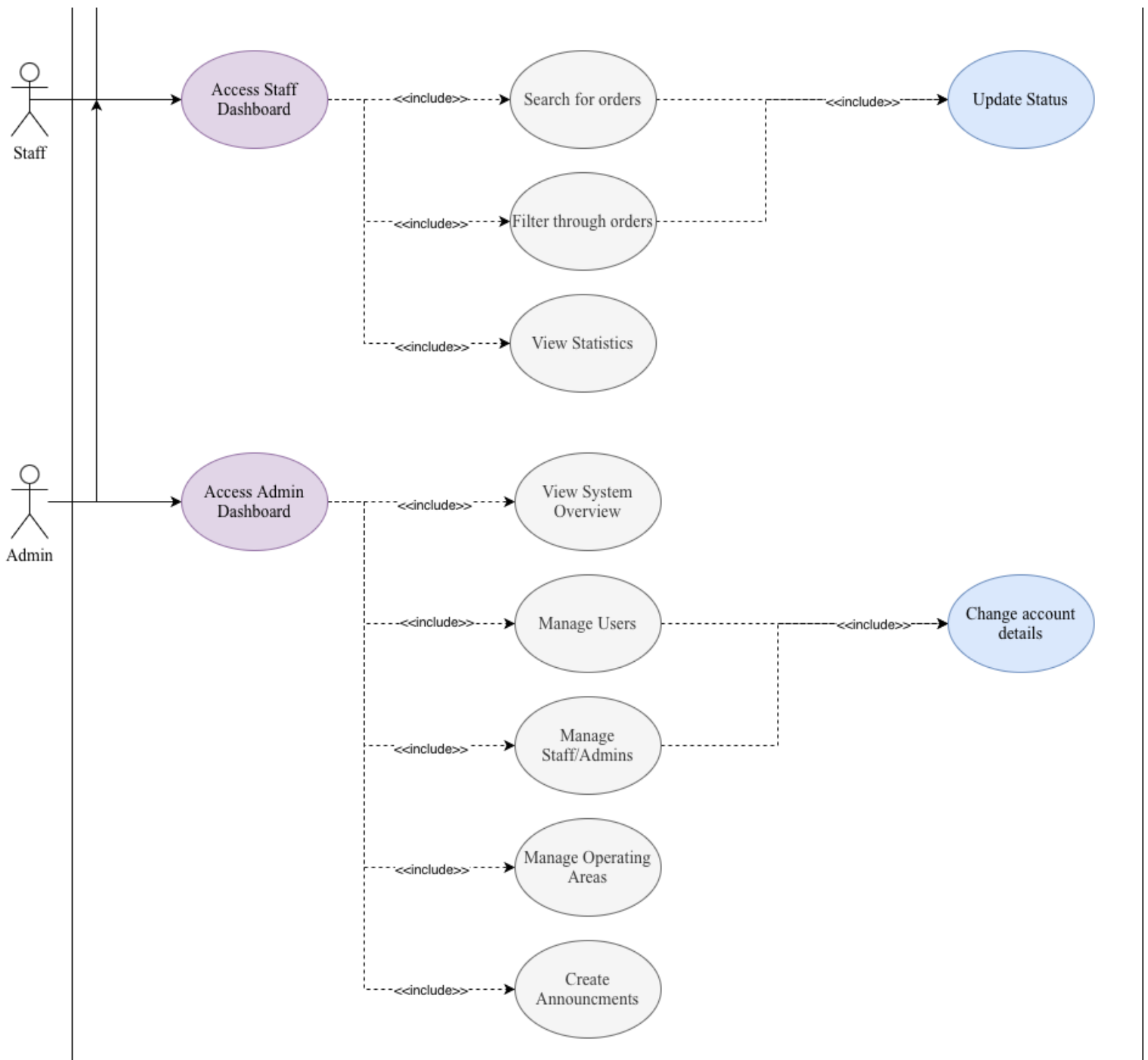


Figure 11: UML Use Case Diagram

3.4 Main System Workflows

The system is organised around several key workflows that represent how different types of users interact with the application. These workflows were designed to reflect real parcel delivery processes while maintaining clarity and usability.

The most important workflow is the parcel booking process. This begins either from the homepage or directly from the send page. The homepage includes a simplified input section that allows users to enter basic details such as postcodes and parcel weight. This information is passed into the main booking form to reduce repeated input.

The main booking process is implemented as a structured multi-section form. All required information is divided into logical sections, including contact details, sender details, recipient details, and parcel information. Each section remains locked until the required inputs in the previous section are completed. This ensures that users follow the correct order while still being able to see the overall process.

Validation is applied throughout the process. Postcodes must match entries stored in the database, ensuring that orders can only be placed within supported operating areas. If invalid postcodes are entered, the user cannot proceed. Additional validation is applied to fields such as names, phone numbers, parcel dimensions, and values. Once all sections are completed, the user is taken to a review page where all entered data can be checked and edited before payment.

The payment workflow is integrated into the booking process. After confirming the order, the user is redirected to Stripe Checkout, where payment is completed securely. Only after a successful payment is the order created in the database and assigned a unique reference number. This prevents incomplete or unpaid orders from being stored.

The tracking workflow is designed to be accessible without requiring authentication. Users can enter a reference number either from the homepage or a dedicated tracking page. Once validated, the system retrieves the corresponding order and displays its current status and tracking history. This provides transparency without requiring account creation.

The staff workflow focuses on operational management of orders. Staff users can search for orders using multiple criteria and filter them based on status. Each order view includes parcel details, tracking history, and controls for updating parcel progress. Status updates are restricted to predefined values to ensure consistency across the system.

Update Parcel Progress

New Status

Awaiting Collection

Location

Select current location

Tracking Update

Describe what changed or what action was taken.

Required when keeping the same status.

Save Update

Figure 12: Staff Dashboard Status Update section

When updating an order, staff must select both a status and a location. Locations are selected from values stored in the database, preventing invalid entries. Additional tracking notes can be added only in case status does not change for appropriate explanation.

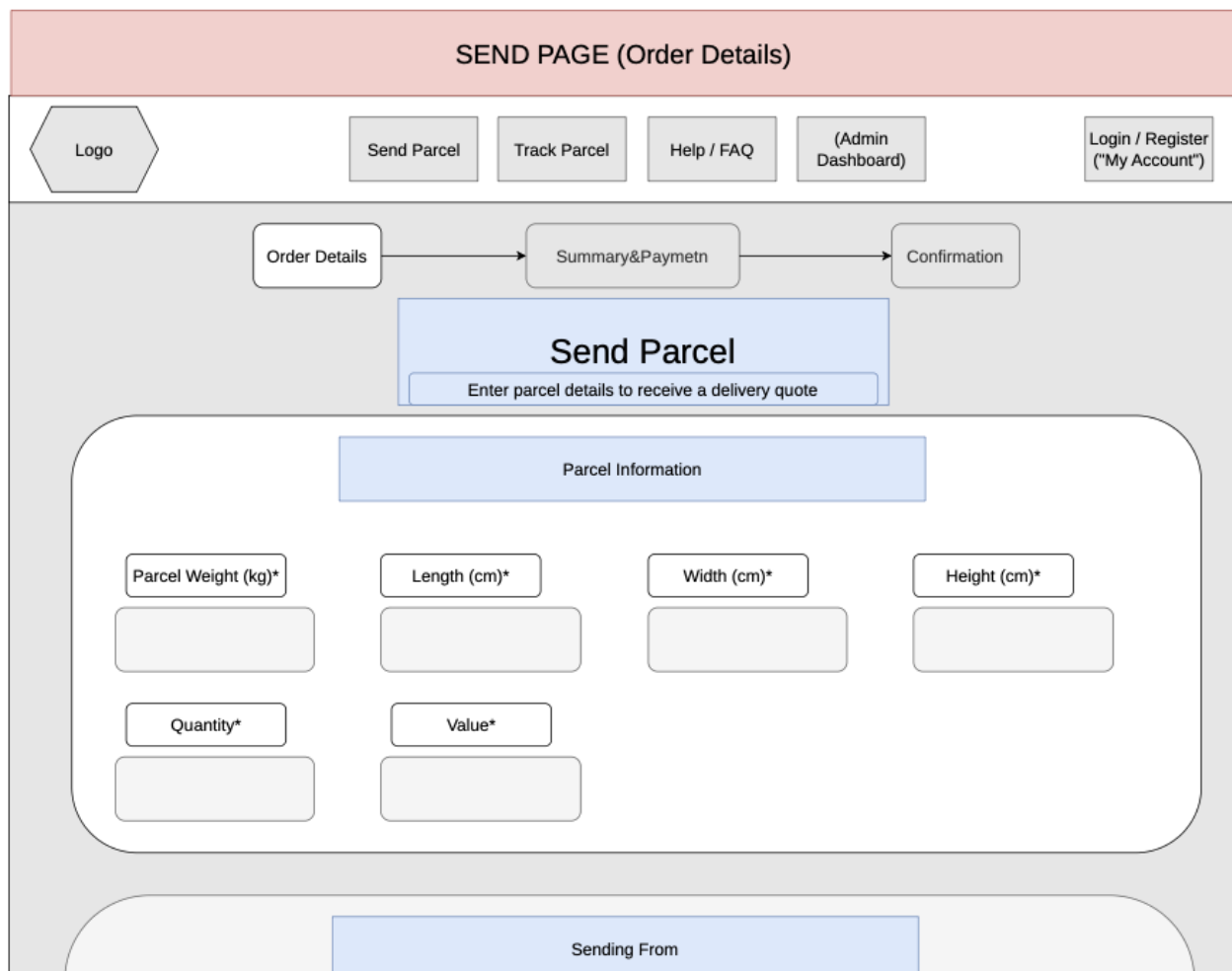
The administrative workflow extends the staff functionality by providing system-level control. Administrators can manage users, assign roles, update operating areas, and create announcements. These features allow the system to be configured without modifying the underlying code.

Overall, the workflows were designed to balance usability, control, and data integrity. Each process ensures that user input is validated, system rules are enforced, and the parcel lifecycle remains consistent from creation to delivery.

3.5 User Interface Design

The user interface was designed with a focus on clarity, responsiveness, and ease of use. The goal was to ensure that users could complete key tasks, such as sending or tracking a parcel, with minimal confusion and without requiring prior experience.

Early in the project, wireframes were created to outline the structure of the system and define how users would interact with different features. These wireframes focused on layout, navigation, and grouping of information rather than visual styling. They provided a starting point for the implementation but were refined during development as usability considerations became clearer.



The wireframe illustrates the 'Send Page' layout, organized into three distinct sections, each with a header bar and a table of input fields.

Sending From

Sending From	
Name*	
Postcode*	
Address Line 1*	
Address Line 2	
City/Town*	

Sending To

Sending To	
Name*	
Postcode*	
Address Line 1*	
Address Line 2	
City/Town*	
Delivery Information	

Collection Option

Collection Option	
☐ Collection from my address	2.50

Figure 13: Send Page Wireframe

During implementation, several changes were made compared to the original design. The most significant changes were applied to the parcel booking process. While the initial design focused on a simpler linear layout, the final version introduced a structured multi-section form with clearer separation between stages and improved validation feedback.

The screenshot displays the 'Send Parcel' page in the ParcelPro system. The page has a light purple header with the 'ParcelPro' logo and navigation links: 'Send Parcel', 'Track Parcel', 'Help / FAQ', 'Staff Dashboard', and 'Admin Dashboard'. A user dropdown menu shows 'Admin User'. Below the header, a progress bar indicates three steps: 1. Order Details (active), 2. Summary & Payment, and 3. Confirmation.

The main heading is 'Send Parcel', followed by the instruction 'Enter parcel details to receive a delivery quote.' The form is divided into several sections:

- Contact Details:** Includes 'Contact Email' and 'Contact Phone*' (with example 'e.g. 07911123456').
- Sender Details:** Labeled 'Complete previous section first'. Includes 'Sender Name*' (filled with 'Admin User'), 'Address Line 1*', 'Address Line 2', 'City*', and 'Postcode*'.
- Recipient Details:** Labeled 'Complete previous section first'. Includes 'Recipient Name*', 'Address Line 1*', 'Address Line 2', 'City*', and 'Postcode*'. It also has a 'Delivery Instructions' text area with a note 'Optional. Maximum 1000 characters.'
- Parcel Details:** Labeled 'Complete previous section first'. Includes 'Weight (kg)*', 'Length (cm)*', 'Width (cm)*', 'Height (cm)*', 'Quantity*' (filled with '1'), and 'Parcel Value (£)*' (filled with '0.00').
- Collection Options:** Labeled 'Complete previous section first'. Includes two radio button options: 'Collection from my address' (costing £2.50) and 'Drop-off at collection point' (Free).

A blue 'Continue to Summary' button is located at the bottom right of the form.

Figure 14: Send Page Final

The final interface improves usability in several ways. Key actions such as sending and tracking parcels are accessible directly from the homepage. The booking process is divided into clear sections, which helps users understand what information is required at each stage. Feedback is provided immediately when inputs are invalid, reducing the likelihood of errors during submission.

Consistency was also an important consideration. The same layout patterns are used across customer pages, staff dashboards, and admin panels. Cards, tables, and form elements follow a consistent style, making the system easier to navigate.

The staff dashboard was designed to support operational tasks efficiently. It includes summary information, filtering options, and structured order views, allowing staff to locate and update orders quickly without unnecessary steps.

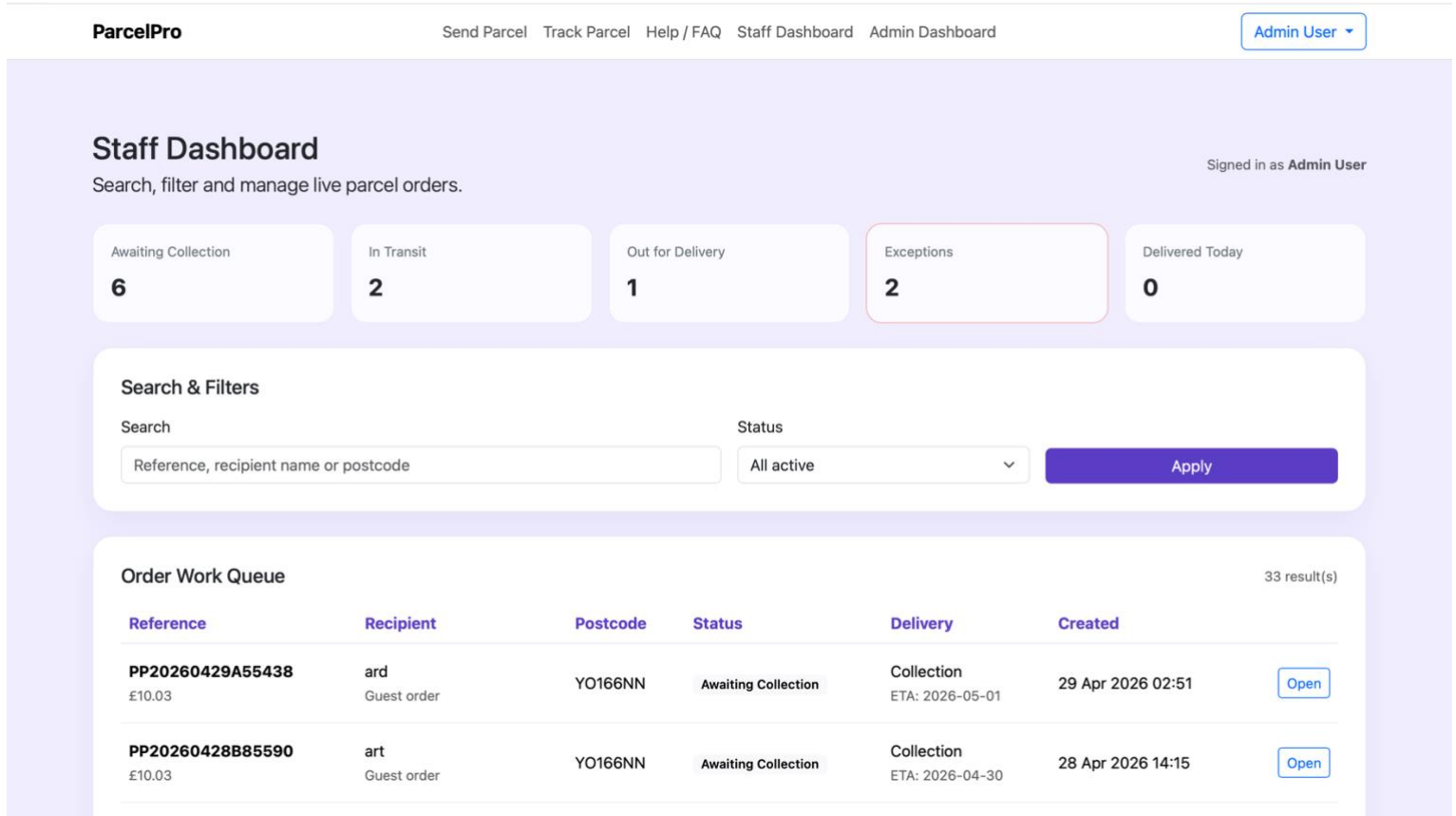


Figure 15: Staff Dashboard

Responsiveness was achieved using Bootstrap, ensuring that the system can be used on different screen sizes. While the application is primarily designed for desktop use, the layout adapts to smaller screens without breaking functionality.

Overall, the interface evolved from an initial structural design into a more refined and user-focused system. The final design reflects both the original planning and the improvements made during implementation.

3.6 Security Design

Security was considered as part of the system design from the beginning rather than being added after implementation.

The design includes multiple layers of validation. Client-side validation improves usability by providing immediate feedback, while server-side validation ensures that invalid or malicious input cannot be processed. This approach ensures that the system remains secure even if client-side controls are bypassed.

Role-based access control is integrated into the design. Different user roles are defined, and access to functionality is restricted accordingly. Staff and admin features are separated from general user functionality, and access is enforced on the server side.

Session management is used to maintain user state and support multi-step processes such as parcel booking. Temporary data is stored securely in the session and cleared when it is no longer required.

The database design also contributes to security by enforcing relationships and restricting invalid data. Sensitive operations, such as updating order status or managing user roles, are only available to authorised users.

Overall, the design ensures that security is built into the structure of the system and supports the implementation measures described later.

Chapter 4 – Implementation and Results

4.1 Homepage, Login and Registration

The homepage acts as the main entry point to the system and is designed to provide immediate access to the two most important actions: sending a parcel and tracking an existing order. These actions are presented prominently through a combined input section, allowing users to begin tasks without navigating through additional pages.

The screenshot displays the ParcelPro homepage. At the top, the logo 'ParcelPro' is on the left, and navigation links 'Send Parcel', 'Track Parcel', 'Help / FAQ', and 'Staff Dashboard' are in the center. A 'Staff User' dropdown menu is on the right. The main heading is 'Send or Track Your Parcel'. Below it is a form with two tabs: 'Send Parcel' (active) and 'Track Parcel'. The form contains three input fields: 'From postcode', 'To postcode', and 'Parcel weight (kg)' with a unit selector. A 'Get Quote' button is on the right. A light blue banner below the form states '10%: Yes - small boxes send cheaper' with a close button. Below the banner is a section titled 'Reliable Parcel Delivery Within Your Area' with a subtext 'Our parcel delivery service ensures safe and reliable transportation of your packages' and an image of hands holding parcels. At the bottom, a dark cookie consent banner reads 'Cookies' and 'We use essential cookies for login/session security and optional cookies to improve your experience.' with 'Reject Optional' and 'Accept All' buttons.

Figure 16: Home page

The homepage includes a simplified input form that allows users to enter postcodes and parcel weight. This data is passed directly to the main booking process, reducing repeated input and improving usability.

Authentication is handled through a combined login and registration flow. When a user enters an email address, the system checks whether the account exists. If the email is found, the user is prompted to enter their password. If the email is not found, the system redirects the user to the account creation process.

The registration process collects name, email, phone number, and password. Password validation enforces minimum length and complexity requirements, including upper and lower case characters and numeric values. Password confirmation is required to prevent input errors.

Once authenticated, users gain access to account-specific features. The navigation bar updates to display the user's name, and a dropdown menu provides access to account management, order history, and logout functionality.

4.2 Parcel Booking Flow

The parcel booking process is implemented as a structured multi-step workflow that collects all required information before allowing the user to proceed to payment.

The process begins with contact details, followed by sender and recipient information, and then parcel details. Each section includes validation rules to ensure that inputs are complete and correctly formatted. For example, postcodes must match values stored in the database, names must contain valid characters, and parcel dimensions must fall within acceptable ranges.

Validation is implemented at three levels:

- HTML validation provides immediate feedback
- JavaScript enhances usability and interaction
- PHP enforces rules on the server side

This layered approach ensures that invalid input cannot bypass the system.

Postcode validation was an important part of the order process because the system must reject parcels outside approved service areas. The postcode format is checked first, then the normalised postcode is compared with active records in the *operating_areas* table. This means a postcode can be syntactically valid but still rejected if the courier company does not currently support that location. The same check is performed on the server to prevent users from bypassing the browser validation.

Validation was applied to almost every user input. Phone numbers are restricted to UK mobile number formats, names must contain realistic name characters and cannot contain numbers, and passwords must meet strength rules including uppercase, lowercase, number, and minimum length requirements. Parcel fields such as weight, dimensions, quantity, and value are restricted to valid numeric ranges. Optional fields are limited to email, address line 2, and delivery instructions. The system also warns users if sender and recipient postcodes are the same, but does not block the order because this may be intentional. If either postcode is not in the active operating areas table, progression is blocked and the user cannot continue to payment.

For logged-in users, an optional feature allows sender details to be automatically filled using a saved address. This reduces repeated input and improves efficiency.

All data entered during the booking process is stored in the session. This allows users to move between sections without losing progress and supports the review/edit functionality before payment.

4.3 Pricing, Delivery Estimates and Reference Numbers

Pricing is calculated dynamically based on parcel characteristics and delivery type.

The total cost includes:

- base price
- weight charge
- size charge
- quantity charge
- optional collection charge

The calculation is performed on the server to ensure consistency and prevent manipulation.

Estimated delivery dates are also calculated automatically. The system adjusts delivery time based on the selected delivery type and excludes weekends to reflect realistic delivery conditions.

```
function estimateDeliveryDate(string $deliveryType): string {  
    $daysToAdd = ($deliveryType === 'collection') ? 2 : 3;  
    $date = new DateTime();  
    while ($daysToAdd > 0) {  
        $date->modify('+1 day');  
        if ((int)$date->format('N') < 6) {  
            $daysToAdd--;  
        }  
    }  
    return $date->format('Y-m-d');  
}
```

Each completed order is assigned a unique reference number. This is generated using a combination of the current date and random values, ensuring that references are both readable and unique.

```
function generateReferenceNumber($conn): string {
    do {
        $reference = 'PP' . date('Ymd') . strtoupper(bin2hex(random_bytes(3)));
        $check = pg_query_params(
            $conn,
            "SELECT 1 FROM public.orders WHERE reference_number = $1 LIMIT 1",
            [$reference]
        );
    } while ($check && pg_num_rows($check) > 0);
    return $reference;
}
```

4.4 Stripe Payment Integration

Stripe was integrated to handle payment processing through a hosted checkout system. After confirming order details, users are redirected to Stripe to complete payment securely.

The system creates a checkout session containing the total price and user details. If the user has already provided an email address, it is passed to Stripe automatically.

Testing was performed using Stripe’s sandbox environment. The card number “4242 4242 4242 4242” can be used to simulate successful transactions.

After payment is completed, Stripe redirects the user back to the system. Only then is the order created and stored in the database. If payment is cancelled or fails, no order is recorded.

This design also reduced the security responsibility of the application, as card details were never handled or stored by ParcelPro.

4.5 Tracking and Order History

The tracking system allows users to view parcel progress using a reference number. Tracking can be accessed from the homepage or a dedicated tracking page.

Once a valid reference is entered, the system retrieves the corresponding order and displays:

- current status
- delivery progress
- tracking history

Tracking history includes timestamps and status updates, providing full visibility of parcel movement.

Registered users also have access to an order history page, where they can view all previously created orders. Each order can be opened to display full details and tracking information.

4.6 Account Management

The account management system allows users to update personal information, change passwords, and store a default sender address.

Users can modify their name, email, and phone number, with validation applied to ensure correct formatting. Password changes require the current password to be entered before a new password is accepted, providing an additional layer of security.

The saved address feature allows users to store a single sender address, which can then be reused during the booking process. This improves usability by reducing repeated input.

4.7 Staff Dashboard

The staff dashboard provides tools for managing parcel delivery progress. It includes a summary of active orders and allows staff to search for specific parcels using reference numbers, names, or postcodes.

Orders can be filtered by status, enabling staff to focus on relevant tasks. Each order view includes parcel details, tracking history, and controls for updating status.

Status updates are restricted to predefined values, ensuring consistency. Staff users cannot modify cancelled orders, and certain actions are limited to administrators.

This was implemented to make the staff interface operational rather than just informational, as staff users can move an order through its lifecycle while maintaining an auditable tracking history.

4.8 Admin Dashboard

The admin dashboard extends system functionality by providing management tools for users, staff, operating areas, and announcements.

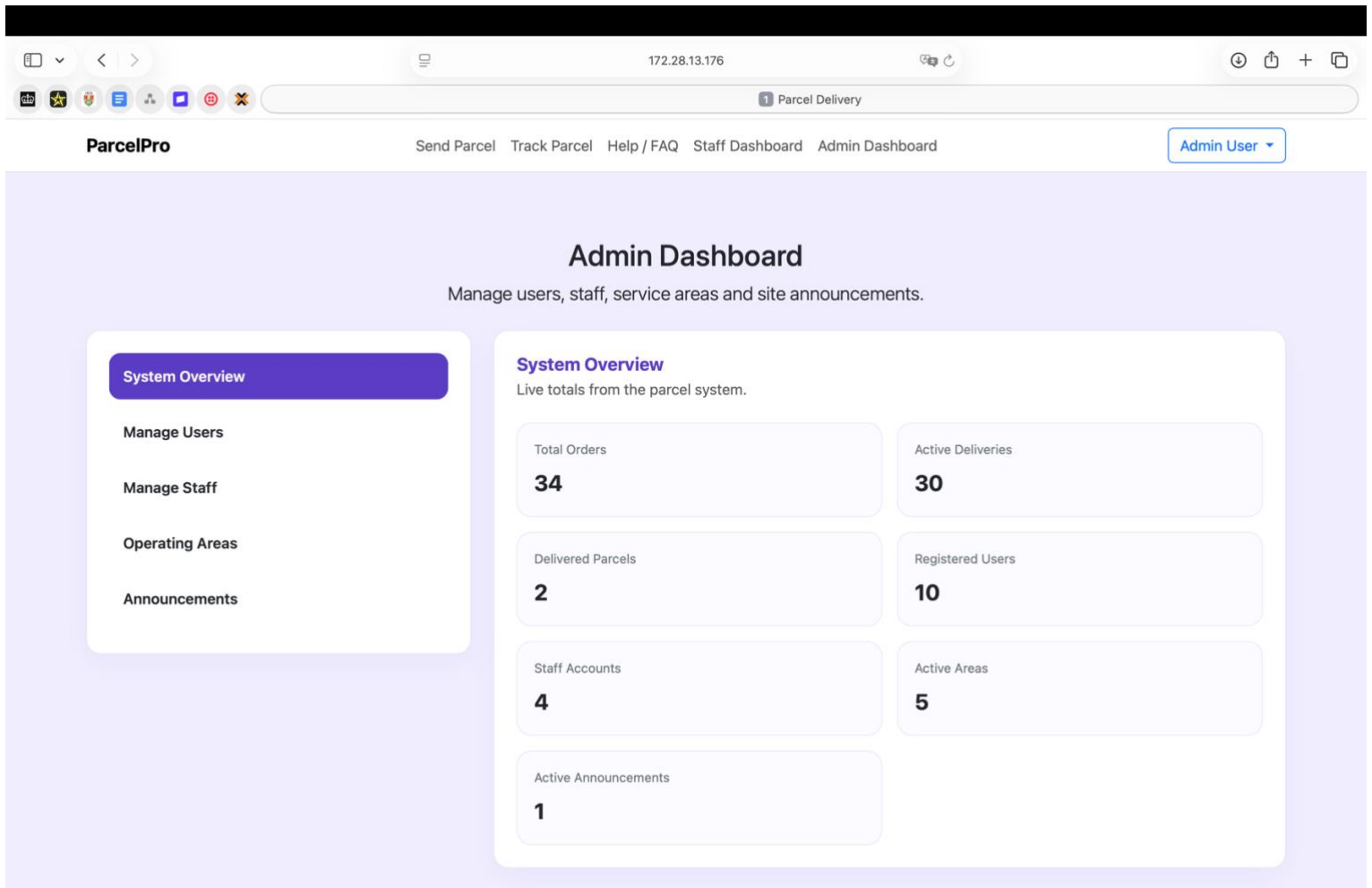


Figure 17: Admin Dashboard

Administrators can:

- search and update user accounts
- assign roles (customer, staff, admin)
- enable or disable accounts
- manage operating areas
- create and manage announcements

Safeguards are implemented to prevent administrators from removing their own access or performing invalid actions.

4.9 Twilio SMS Integration

The system integrates with Twilio to send SMS notifications for key events such as order confirmation and delivery updates.

Each SMS attempt is recorded in the database, including:

- recipient number
- message content
- provider response
- error messages

Order	Provider	Status	Message	Error	Created
31	twilio	failed	ParcelPro: Order PP20260427D4E94C has been placed and paid.	Message cannot be sent with the current combination of 'To' (+44776020XXXX) and/	2026-04-27 01:23
30	twilio	failed	ParcelPro: Your parcel PP20260427620E31 status is now awaiti	Message cannot be sent with the current combination of 'To' (+44776020XXXX) and/	2026-04-27 01:16
30	twilio	failed	ParcelPro: Order PP20260427620E31 has been placed and paid.	Message cannot be sent with the current combination of 'To' (+44776020XXXX) and/	2026-04-27 01:16

Figure 18: Database error logging

Testing revealed limitations due to the Twilio trial environment. Messages sent from a US number to UK numbers result in error code 21612.

Twilio Home

My first Twilio account

Trial: £10.84117 Upgrade

Jump to...

Admin

Notifications

Help

Settings

Profile

Account Dashboard

Develop

Monitor

Logs

Errors

Error logs

Webhook

Messaging

Verify

Calls

Conferences

Call recordings

Call transcriptions

Video rooms

Video recordings

Video compositions

View and inspect error and warning events generated by your account.

Event type

All

Value

Search by error codes or SID

Date/time range

Last 30 days

Apply

Clear all

Maximum 3 queries. Separate each with a comma.

Select view

View all

> Historical trend

Event	Product	Timestamp
21612 - Message cannot be sent with the current combination of "To" and/or "From" parameters	Messaging	12:23am UTC 2026-Apr-27
21612 - Message cannot be sent with the current combination of "To" and/or "From" parameters	Messaging	12:16am UTC 2026-Apr-27
21612 - Message cannot be sent with the current combination of "To" and/or "From" parameters	Messaging	12:16am UTC 2026-Apr-27

Figure 19: Twilio Error Logs

To support demonstration, the system allows SMS functionality to be disabled while still logging attempts.

```
function getSmsConfig(): array
{
    return [
        'account_sid' => '[redacted]',
        'auth_token' => '[redacted]',
        'from_number' => '+1XXXXXXXXXXXX',
        'enabled' => false,
    ];
}
```

Credentials are redacted in this report and should not be committed to a public repository.

Order	Provider	Status	Message	Created
36	system_log	sent	ParcelPro: Your parcel PP20260429A55438 status is now awaiti	2026-04-29 02:52
36	system_log	sent	ParcelPro: Order PP20260429A55438 has been placed and paid.	2026-04-29 02:51
35	system_log	sent	ParcelPro: Your parcel PP20260428B85590 status is now awaiti	2026-04-28 14:21
35	system_log	sent	ParcelPro: Order PP20260428B85590 has been placed and paid.	2026-04-28 14:15
34	system_log	sent	ParcelPro: Order PP202604279DAFB4 has been placed and paid.	2026-04-27 22:56

Figure 20: Database with SMS disabled

This ensures that the full SMS workflow is implemented, even though delivery is restricted in the testing environment.

4.10 Security Implementation

Security controls were implemented across authentication, form handling, database access, and restricted pages. Prepared PostgreSQL queries were used when processing user input, reducing SQL injection risk by keeping user data separate from SQL statements. Dynamic output such as names, addresses, announcements, and tracking descriptions was escaped before display using `htmlspecialchars()`.

Authentication used PHP's password hashing functions. Passwords were stored as hashes and verified during login rather than stored in plain text. After successful authentication, the session ID was regenerated to reduce session fixation risk.

Sensitive forms included CSRF tokens linked to the user session. Requests with missing or invalid tokens were rejected before the requested action was processed. This was applied to

actions such as login, registration, logout, order submission, payment creation, account updates, staff tracking updates, and admin changes.

The application uses essential cookies to support session-based functionality, including maintaining login state, CSRF protection, and temporary order data during the booking process. A cookie banner is displayed to inform users and allow acceptance or rejection of optional cookies. In the current implementation, this preference does not affect core functionality, but provides a foundation for future extensions such as analytics or personalised features.

Access control was enforced server-side. Staff and admin pages used role checks, so restricted functionality could not be accessed simply by entering a URL. Additional checks were also added for sensitive actions, such as preventing staff users from cancelling orders and preventing administrators from disabling their own admin access.

Validation was applied both client-side and server-side. Client-side validation improved usability, but PHP validation remained the final control because browser-side checks can be bypassed.

4.11 Deployment

The system was deployed on a Debian-based university virtual machine using Apache.

The application is accessible at:

<https://debproj-ard38.teachvm.aber.ac.uk/index.php>

Access is restricted to authorised users through the university network or VPN.

The deployment process included:

- database migration
- configuration updates
- Apache setup
- HTTPS configuration using Let's Encrypt

This demonstrates that the system can operate outside a local environment and supports realistic deployment conditions.

4.12 Final Result

The final system provides a complete parcel delivery workflow from order creation to tracking and management.

Customers can submit orders, complete payments, and track parcels. Staff users can manage delivery progress, and administrators can control system configuration.

Despite some limitations related to external services, deployment constraints and access constraints on the virtual machine, the system meets its core objectives and demonstrates a functional and secure parcel management platform.

Chapter 5 – Testing

5.1 Testing Strategy

Testing was carried out throughout development rather than only at the end of the project. Because the system contains several connected workflows, testing focused on confirming that each feature worked independently and also as part of the wider parcel ordering process.

The main testing approach was manual system testing. This was appropriate because many important parts of the application involve full user journeys, such as completing a parcel order, being redirected to Stripe, returning to the confirmation page, and then tracking the parcel using the generated reference number.

Testing was organised around the main requirements of the system. These included registration, login, parcel booking, postcode validation, payment, tracking, account management, staff updates, admin management, and SMS logging. Additional testing was carried out for validation and security behaviour, especially where invalid input or restricted access could affect the system.

While manual testing was effective for validating full workflows, it is less efficient for regression testing. Future improvements could include automated testing of core backend logic, such as validation functions and database operations, using a unit testing framework. However, due to the interactive and multi-step nature of the system, many scenarios would still require manual end-to-end testing.

Full manual test evidence is included in the technical submission. This chapter summarises the testing approach and provides representative test cases.

5.2 Functional Testing

Functional testing checked whether the main features of the system behaved as expected. The most important test was the complete parcel booking workflow. This involved starting from the homepage, entering valid postcodes and parcel weight, completing the send form, reviewing the order, accepting the required conditions, completing Stripe test payment, and receiving a confirmation page with a tracking reference.

Login and registration were also tested. Existing emails correctly opened the password step, while new emails opened the registration form. Registered users were then able to access account management and order history.

The tracking system was tested by entering valid and invalid reference numbers. Valid references opened the order details and tracking history, while invalid formats were rejected before searching.

Staff functionality was tested by logging in as a staff user, searching for an order, opening the order detail page, and updating parcel status. Admin functionality was tested by changing user details, managing staff roles, adding operating areas, and creating announcements.

5.3 Validation Testing

Validation testing was important because the system accepts many user inputs. The send form was tested using both valid and invalid data. Invalid phone numbers, invalid emails, invalid names, missing required fields, invalid parcel dimensions, and unsupported postcodes were tested.

The postcode validation was tested in two stages. First, the format had to match a valid UK postcode pattern. Second, the postcode had to exist as an active record in the *operating_areas* table. This ensured that a postcode could not be accepted simply because it looked valid.

The system was also tested with matching sender and recipient postcodes. In this case, the system displayed a warning but allowed the user to continue. This was intentional because a local delivery or correction may still be valid.

Admin and account forms were also tested with invalid data. Email fields required valid email formats, phone fields required valid UK mobile formats, and postcode fields required valid postcode formats. Dropdown fields were used where possible to reduce invalid input.

5.4 Security Testing

Security testing focused on confirming that the implemented controls behaved correctly.

CSRF protection was tested by submitting forms without a valid token. These requests were rejected. Role-based access control was tested by attempting to access staff and admin pages without the correct role. Unauthorised users were redirected or denied access.

SQL injection resistance was considered by testing form inputs with unexpected characters and SQL-like input. Because database queries use parameterised statements, user input is treated as data rather than executable SQL.

Output escaping was checked by entering characters that could otherwise be interpreted as HTML. The system displayed this content safely rather than rendering it as markup.

Authentication testing confirmed that incorrect passwords were rejected and disabled accounts could not log in. Password change functionality required the current password before a new password could be accepted.

5.5 Example Test Cases

Test ID	Area	Test	Expected Result	Result
TC1	Registration	Register with name, UK phone, email, and strong password	Account is created and user is logged in	Pass
TC2	Registration	Register with weak password	Registration is blocked with validation feedback	Pass
TC3	Parcel booking	Enter valid phone number, operating area postcodes and valid parcel details	User can continue to summary page	Pass
TC4	Parcel booking	Enter postcode not in <i>operating_areas</i>	Continue button remains unavailable / request rejected	Pass
TC5	Parcel booking	Enter same sender and recipient postcode	Warning is shown but user can continue	Pass
TC6	Payment	Complete Stripe test payment using <i>4242 4242 4242 4242</i>	Order is created and confirmation page is shown	Pass
TC7	Tracking	Search using valid reference number	Tracking details and history are displayed	Pass
TC8	Tracking	Search using invalid reference format	Search is rejected with validation feedback	Pass

TC9	Staff	Staff updates parcel status and location	Order status and tracking history are updated	Pass
TC10	Staff	Staff updates parcel status by using the same status	Warning for adding description why status remains the same	Pass
TC11	Admin	Admin adds a new operating area	New postcode becomes available for validation	Pass
TC12	Access control	Customer attempts to access admin dashboard	Access is denied	Pass
TC13	SMS	Trigger order/status notification	SMS attempt is logged in database	Pass / partial delivery

5.6 Testing Limitations

The main limitation of testing was that it was manual rather than automated. This allowed the full workflows to be checked realistically, but it also means regression testing would take longer if the system continued to grow.

Stripe was tested using the sandbox environment, so real payment processing was not used. This was appropriate for the project, but production deployment would require live keys, further payment failure testing, and webhook-based confirmation.

SMS testing was limited by Twilio trial and routing restrictions. The system logs SMS attempts and provider responses, but reliable UK delivery would require a production Twilio setup.

Automated testing frameworks were not used due to the full-system workflow nature of the application, which required manual end-to-end testing of user interactions, payment redirection, and role-based functionality.

No formal user evaluation was completed. The interface was reviewed and refined during development, but future work could include structured user testing with customers and staff to evaluate usability more formally.

Chapter 6 – Critical Evaluation

6.1 Evaluation Against Requirements

The system was evaluated against the functional requirements defined in Chapter 1. Overall, the implementation satisfies the majority of the specified requirements and demonstrates a complete end-to-end parcel management workflow.

Customers are able to register accounts and place parcel orders either as registered users or as guests. The parcel booking process supports structured input of sender, recipient, and parcel details, with validation applied at multiple levels. Postcode validation is enforced through the operating areas table, ensuring that only supported geographical areas are accepted. This directly satisfies the requirement for restricting service availability based on location.

Payment functionality has been successfully integrated using Stripe in test mode. The system ensures that orders are only created after successful payment, preventing incomplete or invalid transactions from being stored. Each order is assigned a unique reference number and includes an estimated delivery date calculated dynamically based on delivery type and working days.

Tracking functionality is fully implemented, allowing users to retrieve parcel information using a reference number without requiring authentication. Tracking history is recorded and displayed, providing visibility into the parcel lifecycle. Staff users are able to update parcel progress through controlled status updates, while administrative users can manage system data, including users, staff accounts, operating areas, and announcements.

The SMS notification requirement is partially satisfied. The system includes a complete integration with the Twilio API and logs all SMS attempts, including provider responses and error messages. However, message delivery is limited by provider restrictions in the testing environment.

Security requirements are addressed through multiple measures, including server-side validation, parameterised queries, output escaping, password hashing, CSRF protection, and role-based access control. These controls ensure that the system operates securely when handling user data and restricted functionality.

Overall, the system meets its core objectives and demonstrates the required functionality in a consistent and reliable manner.

6.2 Strengths

One of the strongest aspects of the system is the user interface and overall user experience. The application was designed with usability as a primary focus, beginning with detailed wireframes and refined during implementation. The final interface presents a clear and structured workflow that allows users to complete tasks without confusion.

The homepage provides immediate access to sending and tracking functionality, reducing unnecessary navigation. The parcel booking process is implemented as a structured multi-section form, where users are guided through the required steps in a logical order. Validation feedback is presented clearly, highlighting errors and preventing incorrect input from progressing. This reduces user frustration and improves completion rates.

Consistency across the interface is another strength. Similar layout patterns are used across customer pages, staff dashboards, and administrative panels, making the system easier to understand and navigate. Visual elements such as progress indicators, status labels, and structured data displays improve clarity and reinforce the system's workflow.

From a technical perspective, the system demonstrates a complete integration of core components. The combination of parcel booking, payment processing, tracking, and administrative management creates a fully functional workflow rather than isolated features. The use of sessions to maintain state across multi-step processes contributes to a smooth user experience.

Validation is implemented thoroughly across the system. Inputs are checked at the client side using HTML and JavaScript, and enforced on the server side using PHP. This layered approach ensures both usability and security. Postcode validation against database records ensures that only valid service areas are accepted, while field-specific validation prevents incorrect data from being submitted.

Security implementation is also a strong aspect of the system. The use of parameterised queries prevents SQL injection, while output escaping reduces the risk of cross-site scripting. Passwords are stored securely using hashing, and session handling includes regeneration after authentication. CSRF protection is applied to sensitive actions, and role-based access control ensures that restricted functionality is protected.

The system also demonstrates realistic integration with external services. Stripe is used to handle payments securely through a hosted checkout, reducing the complexity of handling sensitive payment data. Twilio integration, although limited by external restrictions, demonstrates the full workflow of SMS notification handling, including logging and error management.

Overall, the system combines usability, functionality, and security in a way that reflects a practical and well-structured web application.

6.3 Weaknesses and Limitations

Despite the overall functionality of the system, several limitations remain.

One limitation is the lack of self-service account deletion. While users are able to manage their profile information and passwords, they cannot delete their account directly. This represents a gap in terms of GDPR-related functionality and would require additional implementation to handle data removal safely.

The announcements feature is implemented but not fully integrated across all parts of the system. While announcements can be created and displayed, they are not directly connected to the parcel booking process or payment flow. This limits their effectiveness as a communication tool within the application.

A design limitation exists in the handling of location data for tracking updates. The system currently uses the operating areas table for both postcode validation and tracking location selection. While this approach simplifies the implementation, it does not scale well. In a larger system, a separate structure for operational locations, such as depots or collection points, would be more appropriate.

The SMS notification system is affected by external constraints. While the integration is technically complete, message delivery is restricted due to provider limitations in the testing environment. As a result, SMS functionality cannot be demonstrated fully in practice, although all attempts are logged correctly.

Access to the deployed system is also limited. The application is hosted on a university virtual machine and can only be accessed through the university network or VPN. This restricts broader usability and prevents public testing.

In addition, some areas of the administrative interface could be improved in terms of feedback and error handling. While validation is present, certain actions rely on users entering correct data without extensive guidance, which could affect usability in more complex scenarios. These limitations do not prevent the system from functioning, but they highlight areas where the application could be extended or improved in a more complete production environment.

6.4 SWOT Analysis

The system can be evaluated further using a SWOT analysis.

The primary strengths of the system include its clear user interface, structured workflows, and complete integration of key features such as booking, payment, tracking, and administrative control. The layered validation and security measures also contribute to a reliable and robust application.

Weaknesses are mainly related to incomplete features and design trade-offs. These include the lack of account deletion, partial integration of announcements, and the use of a single table for both operating areas and tracking locations.

Opportunities for improvement include extending the system to support full SMS delivery, implementing account deletion and password reset functionality, and improving scalability by separating operational location data. The system could also be expanded to include more advanced features such as route optimisation, delivery scheduling, or improved reporting for staff and administrators.

Threats are primarily external. These include reliance on third-party services such as Stripe and Twilio, which may introduce limitations or require configuration changes. Deployment restrictions, such as network access limitations, also affect how the system can be used and evaluated.

Overall, the SWOT analysis shows that the system is strong in its core functionality and design, with clear opportunities for future development.

6.5 Future Work

Several areas have been identified for future improvement and extension.

The most immediate enhancement would be the implementation of account deletion functionality. This would allow users to manage their data more fully and improve compliance with data protection expectations.

Another important addition would be a password reset feature. This would require integration with email or SMS verification and would improve account accessibility and security.

The SMS system could be extended by using a production-ready Twilio configuration or an alternative provider that supports reliable delivery to UK numbers. This would allow the notification system to operate fully as intended.

The database design could also be improved by separating tracking locations from operating areas. Introducing a dedicated table for depots or collection points would make the system more scalable and better aligned with real-world courier operations.

The announcements system could be expanded to integrate with more parts of the application, such as displaying relevant messages during parcel booking or payment stages.

Finally, further improvements could be made to the administrative interface, including better feedback, validation messaging, and usability enhancements for managing system data.

These improvements would build on the existing system and move it closer to a production-level application.

Chapter 7 – Conclusion

This project set out to design and implement a web-based parcel management system that supports both customer-facing functionality and internal courier operations. The final system, ParcelPro, demonstrates a complete workflow covering parcel booking, payment processing, tracking, and administrative management within a single integrated platform.

The system successfully allows users to create parcel delivery requests either as guests or registered users, with a structured booking process that ensures data is validated and consistent. The integration of Stripe provides a secure and realistic approach to payment handling, while the tracking system enables users to monitor parcel progress using a unique reference number. Staff and administrative users are provided with dedicated dashboards that support operational tasks, including updating delivery status and managing system data.

A key focus of the project was usability. The interface was designed to guide users through processes clearly and reduce the likelihood of errors. Features such as progressive form sections, validation feedback, and editable review stages contribute to a smooth and structured user experience. This ensures that the system is not only functional, but also practical to use.

Security was also considered throughout the development process. The system implements multiple protective measures, including server-side validation, parameterised database queries, output escaping, password hashing, CSRF protection, and role-based access control. These measures help ensure that user data is handled safely and that restricted functionality is properly protected.

Although some limitations remain, including some external service and deployment constraints and incomplete account management features, the system still demonstrates all core functionality required for a parcel delivery platform. These limitations are primarily due to external constraints or time considerations rather than fundamental design issues.

Overall, the project achieves its main objectives and provides a working example of a modern web-based parcel management system. It demonstrates both technical implementation and practical design considerations, showing how multiple components can be combined into a coherent and usable application.

Annotated References

Stripe Documentation – <https://stripe.com/docs>

Used to implement the payment processing system. Provided guidance on creating checkout sessions, handling redirects, and testing transactions using the sandbox environment.

Twilio Documentation – <https://www.twilio.com/docs>

Used to integrate SMS notification functionality. Documentation supported API usage, message sending, and understanding provider responses and error codes.

PHP Manual – <https://www.php.net/manual/en/>

Referenced extensively for backend implementation, including session management, password hashing (*password_hash*, *password_verify*), and secure coding practices.

PostgreSQL Documentation – <https://www.postgresql.org/docs/>

Used for database design, query structure, and understanding relational constraints and parameterised queries.

Bootstrap Documentation – <https://getbootstrap.com/docs/>

Used to design the user interface and implement responsive layouts, form validation styling, and consistent UI components.

MDN Web Docs (Mozilla Developer Network) – <https://developer.mozilla.org/>

Referenced for JavaScript validation, HTML form behaviour, and general frontend development practices.

Royal Mail Website – <https://www.royalmail.com/>

Analysed as part of the competitor investigation to understand structured parcel booking workflows and service presentation.

Evri Website – <https://www.evri.com/>

Used as a primary reference for homepage design and progressive form behaviour.

DPD Website – <https://www.dpd.co.uk/>

Analysed for tracking functionality and detailed delivery updates, which influenced the tracking system design.

Appendix A – Use of Third-Party Code and Libraries

This project makes use of several third-party tools and libraries to support development and extend system functionality.

Bootstrap

Bootstrap was used to implement the frontend layout and styling. It provided responsive design components, form validation styles, and consistent UI elements across the application. Custom CSS was applied where necessary to adapt the interface to the system's specific requirements.

Stripe API

Stripe was used for payment processing through a hosted checkout system. The integration allows users to complete transactions securely without the application directly handling sensitive card data. The implementation follows Stripe's official documentation and uses the test environment for demonstration.

Twilio API

Twilio was used to implement SMS notification functionality. The system sends messages when key events occur, such as order confirmation and status updates. All SMS attempts are logged in the database, including provider responses and errors. Message delivery is limited in the testing environment.

PHP Built-in Functions and Security Features

The system uses PHP's built-in functionality for password hashing (*password_hash*, *password_verify*), session management, and secure request handling. These features are part of the standard PHP library and do not require external dependencies.

No third-party code was directly copied into the project. External resources were used for guidance and implementation support, and all final code was written and adapted as part of this project.

Apache

Apache was used as the web server to host the application on a Debian-based virtual machine. It handled HTTP requests, routing, and integration with PHP for server-side processing.

PostgreSQL

PostgreSQL was used as the relational database system to store all application data, including users, orders, payments, and tracking history. Its support for constraints and relational integrity ensured consistency across the system.

Bootstrap Icons / UI assets

Bootstrap Icons and standard UI components were used to enhance visual clarity and improve usability within the interface.

Appendix B – Database Tables

users table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Unique user ID
full_name	varchar(100)		User name
email	varchar(150)	UNIQUE, NOT NULL	Login email
phone	varchar(20)		Contact number
password_hash	varchar(255)	NOT NULL	Secure hashed password
role	varchar(20)	customer/staff/admin	Access level
created_at	timestamp	default current timestamp	Registration date
is_active	boolean	default true	Account enabled/disabled
updated_at	timestamp		Last update

user_addresses table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Address ID
user_id	integer	FK → users(id), ON DELETE CASCADE	Linked account
address_line1	varchar(150)	NOT NULL	Main address
address_line2	varchar(150)		Optional address
city	varchar(100)	NOT NULL	City
postcode	varchar(20)	NOT NULL	Postcode
is_default	boolean	default false	Default address marker
created_at	timestamp	default now()	Date added

orders table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Order ID
reference_number	varchar	UNIQUE	Tracking reference
user_id	integer	FK → users(id), ON DELETE SET NULL	Registered customer, if used
sender_name	varchar(100)	NOT NULL	Sender name
sender_address1	varchar(150)	NOT NULL	Sender address
sender_address2	varchar(150)		Optional sender address
sender_city	varchar(100)	NOT NULL	Sender city
sender_postcode	varchar(20)	NOT NULL	Sender postcode
recipient_name	varchar(100)	NOT NULL	Recipient name
recipient_address1	varchar(150)	NOT NULL	Recipient address
recipient_address2	varchar(150)		Optional recipient address
recipient_city	varchar(100)	NOT NULL	Recipient city
recipient_postcode	varchar(20)	NOT NULL	Recipient postcode
delivery_instructions	text		Optional delivery notes
weight	numeric(6,2)	0.10–999.99	Parcel weight
length	numeric(6,2)	1–999.99	Parcel length
width	numeric(6,2)	1–999.99	Parcel width
height	numeric(6,2)	1–999.99	Parcel height
quantity	integer	1–20, default 1	Number of parcels
parcel_value	numeric(10,2)	0–50000	Declared value
delivery_type	delivery_type_enum	NOT NULL	Collection/drop-off
delivery_price	numeric(10,2)	>= 0	Delivery charge
collection_price	numeric(10,2)	>= 0	Collection charge
total_price	numeric(10,2)	>= 0	Final total
current_status	order_status_enum	default order_placed	Current parcel status
estimated_delivery_date	date		Estimated delivery
created_at	timestamp	default now()	Order date
collection_point_id	integer	FK → collection_points(id), ON DELETE SET NULL	Drop-off point
contact_email	varchar(150)		Contact email

contact_phone	varchar(20)	NOT NULL	SMS/contact phone
payment_status	payment_status_enum	default pending	Payment state
updated_at	timestamp		Last order update

payments table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Payment ID
order_id	integer	UNIQUE, FK → orders(id), ON DELETE CASCADE	Linked order
payment_method	varchar(50)	NOT NULL	Payment method
amount	numeric(10,2)	>= 0	Payment amount
payment_status	payment_status_enum	default pending	Payment state
transaction_reference	varchar(100)		Stripe/session reference
created_at	timestamp	default now()	Payment date

tracking_history table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Tracking entry ID
order_id	integer	FK → orders(id), ON DELETE CASCADE	Linked order
status	order_status_enum	NOT NULL	Parcel status
location	varchar(150)		Current location
description	text		Status note
updated_by	integer	FK → users(id), ON DELETE RESTRICT	Staff/admin who updated
created_at	timestamp	default now()	Update time

sms_notifications table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	SMS log ID
order_id	integer	FK → orders(id), ON DELETE CASCADE	Related order
phone	varchar(20)	NOT NULL	Recipient phone
message	text	NOT NULL	SMS content
provider	varchar(50)	default twilio	SMS provider/log type
provider_sid	varchar(100)		Provider reference
status	varchar(50)	default queued	Sent/failed/logged state
error_message	text		Error details
created_at	timestamp	default now()	Log time

operating_areas table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Area ID
postcode	varchar(20)	UNIQUE, NOT NULL	Approved postcode
city	varchar(100)	NOT NULL	City/town
is_active	boolean	default true	Area enabled/disabled
created_at	timestamp	default now()	Date added

collection_points table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Collection point ID
name	varchar(100)	NOT NULL	Location name
address_line1	varchar(150)	NOT NULL	Address
address_line2	varchar(150)		Optional address
city	varchar(100)	NOT NULL	City
postcode	varchar(20)	NOT NULL	Postcode
is_active	boolean	default true	Available/unavailable
created_at	timestamp	default now()	Date added

announcements table – [back to section 3.2](#)

Field	Type	Key / Constraint	Purpose
id	integer	PK	Announcement ID
title	varchar(150)	NOT NULL	Announcement title
message	text	NOT NULL	Message body
is_active	boolean	default true	Visibility status
created_at	timestamp	default now()	Date created
created_by	integer	FK → users(id), ON DELETE RESTRICT	Admin creator
expires_at	timestamp		Expiry date

Document Change History

Version	Date	Changes made to the document	Changed by
1.0	26/04/2026	Creation of document. Draft version	ard38
2.0	30/04/2026	Rewritten and upgraded version	ard38